Notebook: knn/after_eda_knn_on_breast_cancer.ipynb

[--- CODE CELL ---]

```
from sklearn.datasets import load_breast_cancer
```

[--- CODE CELL ---]

```
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns = data.feature_names)
df['target'] = data.target
# we set target variables as 0 = maligant, 1 is benin
```

[--- CODE CELL ---]

```
df.head()
#it is different than the data we imported from kaggle
```

[--- CODE CELL ---]

```
df.shape
# shows how many rows and coulms we have
```

[--- CODE CELL ---]

```
df.columns
#describes the features of the data
```

[--- CODE CELL ---]

```
df.info()
```

[--- CODE CELL ---]

```
df.isnull().sum()
```

[--- CODE CELL ---]

```
#the above data shows that there exists no missing value present
```

[--- CODE CELL ---]

```
df.describe()
#we can see that all the feauters described have a different valye attacted to it... the mean standard deviation etc chanegs a lot
```

[--- CODE CELL ---]

```
df['target'].value_counts()
```

[--- CODE CELL ---]

```
df['target'].value_counts().plot(kind = 'bar')
```

[--- CODE CELL ---]

```
features = ['mean radius', 'mean texture', 'mean smoothness']
#uni variate analysis that is it takes one feature at a time and explores them
```

[--- CODE CELL ---]

```
df[features].hist()
```

[--- CODE CELL ---]

```
import matplotlib.pyplot as plt
```

[--- CODE CELL ---]

```
plt.figure(figsize=(10,5))
df[features].boxplot()
plt.show()
```

[--- CODE CELL ---]

```
#it shows that there are outliers in the data
```

[--- CODE CELL ---]

```
import seaborn as sns
for feature in features:
    sns.boxplot(x='target', y=feature, data=df)
    plt.title(feature)
    plt.show()
```

[--- CODE CELL ---]

```
sns.scatterplot(x='mean radius', y = 'mean texture', data = df)
plt.show()
```

[--- CODE CELL ---]

```
#correaltion analysis
corr = df.drop('target', axis = 1).corr()
plt.figure(figsize=(12,10))
sns.heatmap(corr, cmap = 'coolwarm')
plt.show()
```

[--- CODE CELL ---]

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
#we need accuracy score to predict if the values are correct or not
#the same formula that we have for True_positive and True_negative
```

[--- CODE CELL ---]

```
X = df.drop('target', axis = 1)
y = df['target']
```

[--- CODE CELL ---]

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

[--- CODE CELL ---]

```
knn = KNeighborsClassifier(n_neighbors = 5)
knn.fit(X_train, y_train)
```

[--- CODE CELL ---]

```
y_pred = knn.predict(X_test)
accuracy_score(y_test, y_pred)
```

[--- CODE CELL ---]

```
#all the accuracies caluculated
# accuracy 1: calculated was: 0.956140350877193
```

[--- CODE CELL ---]

```
# Now we scale the data
# Scaling  why is it done
# Bringing all features to a similar numerical range
# 4. WHY SCALING IS REQUIRED (CORE CONCEPT)
# ------------------------------------------------------------
# KNN uses Euclidean distance:
# distance = sqrt( (x1 - y1)^2 + (x2 - y2)^2 + ... )
# If one feature has much larger values, it dominates the distance.
# Example (WITHOUT scaling):
# Difference in mean radius = 2     -> 2^2 = 4
# Difference in area worst  = 500   -> 500^2 = 250,000
# Result:
# - 'area worst' dominates distance
# - 'mean radius' is almost ignored
# This happens EVEN IF radius is very informative.
# # Scaling fixes this by bringing all features to a similar range.
# What scaling fixes?
# Makes each feature contribute fairly to distance
# Prevents one feature from overpowering others
# Allows KNN to use actual patterns, not raw magnitude
# 5?? Why scaling is CRITICAL for the breast cancer dataset
# From your EDA, you saw:
# 30 features
# Huge variation in ranges
# Many correlated features
# Without scaling:
# Large features dominate
# Small but informative features are ignored
# Distance becomes meaningless
# ? This is why accuracy jumps after scaling.
```

[--- CODE CELL ---]

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

[--- CODE CELL ---]

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[--- CODE CELL ---]

```
knn = KNeighborsClassifier(n_neighbors = 5)
knn.fit(X_train_scaled, y_train)
```

[--- CODE CELL ---]

```
y_pred = knn.predict(X_test_scaled)
accuracy_score(y_test, y_pred)
```

[--- CODE CELL ---]

```
#all the accuracies caluculated after scaling
# accuracy 1: calculated was: 0.9473684210526315
```

[--- CODE CELL ---]